

CURSOR BUILDATHON · PROJECT SUBMISSION DOCUMENT

CURSOR × TECHTALK360 · Confidential — For Authorized Use Only

Field	Value
Project	Cyber League
Track	Best use of n8n
Team	Hackers League
Demo URL	https://youtu.be/d1xiZMBJ61k
Repository	https://github.com/Deadsecnote1/cyberleague (branch <code>cyber - league</code>)

Field	Value
Prepared by	Thanushiyan Kanthasami

Section 01 — Project Overview

Project Name & One-Line Pitch

Cyber League — A local command center that triggers an **n8n-orchestrated** web pentest pipeline with configurable scope, SQL injection checks, rate-limit protection, and optional OpenAI hunt advisor — for **authorized** bug bounty and lab testing.

Summary

Security testers run subdomain enumeration, HTTP probing, template scanners, and SQLi checks across disconnected terminals and scripts, often without scope discipline or rate awareness. **Cyber League** unifies these steps behind one **n8n workflow**: a FastAPI portal collects target URL, scan preset, and pentest scope, then triggers n8n via webhook. n8n executes `bugbounty-scan.sh`, which runs industry tools (subfinder, httpx, ffuf, nuclei, nmap, nikto) plus a dedicated SQLi phase. Findings aggregate into `vulnerabilities.json` and HTML reports; optional AI summarizes priorities before and after the scan. The system targets Parrot/Kali-style Linux workstations and refuses hosts outside user-defined scope.

Submission Details

Item	Detail
Track	Best use of n8n

Item	Detail
k	
Tracking integration	Webhook + form triggers, Code nodes (parse portal payload), Execute Command (<code>bugbounty-scan.sh</code>), AI nodes (pre/post scan), workflow patch tooling (<code>scripts/patch_*.py</code>), SQLite-backed n8n executions
Cursor integration	Portal, workers, n8n JSON export, patches, ops scripts (<code>cyberleague.sh</code> , <code>shutdown.sh</code>), and this submission built iteratively in Cursor
Team name	Hackers League
Demo URL	https://youtu.be/d1xiZMBJ61k
Repository	https://github.com/Deadsecnote1/cyberleague

Item	Detail
Why	

Section 02 — Problem Statement

The Problem

Who: Junior-to-mid bug bounty hunters and web pentest students on Linux who run recon regularly.

Context: On every web engagement, testers repeat subdomain discovery, live-host probing, header checks, template scanning, and SQLi triage—switching tools manually. Without scope rules, subdomain enumeration hits **out-of-scope** hosts. Without rate awareness, scanners trigger WAF blocks and waste program time.

Consequence if unsolved: Missed coverage, program violations, IP blocks that halt testing, and no single audit trail tying configuration to artifacts.

Why It Matters

Dimension	Detail
Frequency	Daily for active bounty hunters; weekly for consultancies and lab courses
Severity	Direct impact on program standing, finding quality, and time-to-report
Current workarounds	Ad-hoc shell scripts, Burp, scattered notes—no orchestration or guardrails
Why workarounds fail	Tools are powerful but not wired to program scope, rate limits, and one reproducible pipeline

Root Cause

Pentest CLIs are mature, but **orchestration with safety defaults** (scope, guards, structured output) is missing. Visual workflow engines like n8n are rarely connected to real recon toolchains with enforceable boundaries.

Section 03 — Proposed Solution

What the Product Does

Cyber League is a browser-based command center on `127.0.0.1:8765`. The user configures target URL, scan intensity (Quick / Standard / Deep), pentest scope (in/out lists or templates), and toggles for SQLi and AI. On submit, the portal validates scope and POSTs JSON to n8n. n8n runs the published **Bug Bounty Pentest Pipeline**, which executes the worker script, aggregates findings, builds HTML reports, and optionally calls OpenAI. The dashboard lists runs with severity counts, blocked status, and report links.

Core mechanism: *Workflow-orchestrated pentest* — the portal is control plane only; **n8n** is the sole scan engine; bash/Python workers perform tool execution.

Key Features (built & demonstrable)

Feature	User need addressed
n8n webhook + form fallback	Reliable engine integration for the n8n track
Quick / Standard / Deep presets	Time-boxed scans vs deep assessment
Pentest scope (3 modes + JSON templates)	Stay within program boundaries
SQL injection phase (<code>scan_sqli.py</code>)	Surface sqli templates + safe probes early
Rate-limit / IP-block guard	Stop when target blocks the scanner
Unified <code>vulnerabilities.json</code> +	Single structured view of

Feature	User need addressed
HTML report	findings
AI pre/post-scan advisor	Prioritization and narrative summary
Delete scan run (UI + API)	Lab cleanup and artifact removal
<code>cyberleague.sh / shutdown.sh</code>	One-command start/stop for demo

Scope

In scope (this build): Single-target web recon on Linux; n8n-orchestrated pipeline; local portal; file-based scans/ and reports/; authorized/lab targets only.

Out of scope (deliberate): Cloud SaaS hosting; multi-tenant auth; authenticated crawling; full sqlmap exploitation; Windows-native install; Docker delivery (GitHub-only for buildathon).

Section 04 — Functional Requirements

Must Have (all demonstrable in demo)

ID	Requirement
M1	User submits target URL and preset from portal; scan starts via n8n
M2	n8n workflow executes <code>bugbounty-scan.sh</code> with profile env vars
M3	Light scan produces subdomain list, live hosts, header/TLS notes
M4	Findings aggregated to <code>vulnerabilities.json</code> and HTML report

I D	Requirement
M 5	Portal dashboard lists runs with severity breakdown
M 6	Pentest scope validated before scan; hosts filtered during scan
M 7	Rate-limit guard can halt scan and record blocked state

Should Have

I D	Requirement
S 1	SQL injection checks (nuclei sqli tags + safe probes)
S 2	OpenAI pre/post-scan advisor (graceful skip without API key)
S 3	Deep profile (nmap, nuclei, nikto per host)
S 4	Delete run removes scans/ workdir and stamp-matched reports
S 5	n8n Executions visible for audit

Could Have / Won't Have (this build)

I D	Item	Decision
C 1	Docker Compose	Won't — GitHub-only delivery
C 2	Public cloud demo	Won't — local 127.0.0.1 only

I D	Item	Decision
	URL	
C 3	Burp export	Could — future
C 4	Multi- target CSV from portal	Partial — n8n CSV path exists; portal focuses single URL

Section 05 — Non-Functional Requirements

Performance

Interaction	Target
Quick preset on lab target	~5–15 minutes (5 hosts, light profile)
Portal dashboard load	< 2s (local filesystem)
n8n webhook response	Immediate (onReceived); long work in execution

Reliability & Error Handling

- Missing CLI tools logged in `tools_missing`; scan continues where possible.
- OpenAI failures skip AI nodes without failing the workflow.
- Webhook 404 → portal falls back to n8n form trigger.
- Invalid scope → portal error; no scan started.
- `.scan_stop` + `emit_final_json` on guard trip or runtime cap.

Usability

- Dark-themed portal; presets with ETA hints; scope section with loadable templates.
- Local-only binding reduces accidental network exposure.

- Typography and layout tuned for readable operator UI.

Scalability

- Filesystem storage suits single analyst today.
- Scale path: object storage for scans/, queue-backed workers, horizontal n8n workers, shared scope service.

Section 06 — Technical Architecture

System Overview

Three layers: **Portal** (FastAPI UI), **Engine** (n8n workflow), **Workers** (bugbounty-scan.sh + Python). Portal never shells out to scanners; it POSTs to n8n only.

Cyber League Portal (:8765)

| webhook JSON + sidecar files



n8n – Bug Bounty Pentest Pipeline

| Parse → AI (optional) → Execute Command



```
bugbounty-scan.sh → subfinder, httpx, ffuf, nuclei, nmap, nikto,  
scan_sqlmap.py
```



```
scans/<domain>_<UTC>/ + reports/*.html
```

See [ARCHITECTURE.md](#) for Mermaid diagram and stack table.

Technology Stack

Layer	Technology	Rationale
Orchestration	n8n	Visual workflow, webhooks, Execute Command, AI nodes, exportable JSON
Portal	FastAPI, Jinja2, uvicorn	Fast local UI; scope validation before engine
Workers	Bash, Python 3	Glue for pentest CLIs and aggregation
Scanners	subfinder, httpx, ffuf, nuclei, nmap, nikto, curl	Industry-standard recon stack

Layer	Technology	Rationale
AI	OpenAI API (gpt-4o-mini default)	Pre/post scan prioritization
Storage	Local filesystem	Simple hackathon/demo deploy on Linux

Data Flow (core action: start scan)

1. User POSTs `/scan` with `target_url`, `scan_config`, `scope`.
2. Portal writes `.current_scan_domain`, `.current_scan_env`, `.current_scan_scope.json`.
3. Portal POSTs n8n webhook `portal-scan` (or form fallback).
4. n8n Parse node reads payload; optional AI pre-scan node runs.
5. Execute Command invokes `bugbounty-scan.sh` with env profile.
6. Worker writes `scans/<domain>_<timestamp>/`; `aggregate_vulnerabilities.py` → `vulnerabilities.json`.
7. n8n report node writes `reports/bugbounty-report-*.html`; optional AI post-scan.

AI Integration

Use Case	Model / method	Role
Pre-scan	OpenAI via <code>ai_pentest_advisor.py</code>	Focus areas before tools run

Use	Model / method	Role
Post-scan	Same	Executive summary merged into report
Failures	Graceful skip	Workflow continues if key missing or API errors
mode		

Known Technical Limitations

- Linux-only workers; some n8n patches use configurable R00T paths.
- Workflow must be **Active** after DB patches (`patch_portal_webhook.py`, etc.).
- SQLi detection ≠ confirmed exploitation.
- Single-machine filesystem; no built-in multi-user auth.

Section 07 — Security

Authentication & Authorisation

A	
c	
t	Mechanism
o	
r	
O	
p	
e	
r	Portal binds to 127.0.0.1 only; no remote access
a	by default
t	
o	
r	
n	
8	
n	
a	Assumed trusted local operator; n8n login if
d	exposed
m	
i	
n	

No student-style multi-tenant auth—single-operator local security tool.

Data Handling

- Scan artifacts under `scans/` and `reports/` on the host.
- No application PII collection; user-supplied targets only.
- Delete run removes `workdir` and stamp-matched reports.

API & Secret Management

- `OPENAI_API_KEY` in `config/openai.env` (gitignored; example file uses placeholder only).
- Keys not exposed in portal frontend; AI runs in `n8n/worker` context.
- `n8n` env: `NODES_EXCLUDE='[]'`, `NODE_FUNCTION_ALLOW_BUILTIN` for required Code node builtins.

Input Validation

- Path traversal blocked on `run_id` and report filenames.
- Scope validation on target URL and host lists before engine start.
- n8n Code nodes validate required `target_url`.

Known Vulnerabilities or Gaps

Gap	Mitigation plan
No TLS on local HTTP	Acceptable for localhost; do not expose portal to LAN without reverse proxy + TLS
n8n default auth if network-exposed	Firewall; bind locally
Automated scanning risk on wrong target	User responsibility; scope + authorized-use warnings in UI/docs

Section 08 — User Stories & Use Cases

Core User Stories

1. As a **tester**, I want to start a quick recon scan from a browser, so that I get subdomains and headers in one step.
2. As a **tester**, I want n8n to orchestrate tools, so that I can audit and extend the pipeline visually.
3. As a **tester**, I want to define in-scope hosts, so that I do not scan out-of-scope subdomains.
4. As a **tester**, I want SQLi checks included, so that obvious injection issues surface early.
5. As a **tester**, I want the scan to stop if the target rate-limits me, so that I do not burn the program IP.
6. As a **tester**, I want an HTML report, so that I can share results with my team.
7. As a **tester**, I want AI to summarize findings, so that I know what to validate first.
8. As a **tester**, I want to delete old lab runs, so that my disk stays clean.

Primary Use Case Walkthrough

Step

User action

System action

1	Opens New scan , enters <code>http://testphp.vulnweb.com</code> , loads vulnweb scope template, selects Quick , enables SQLi + AI	Portal validates scope
2	Clicks start	POST webhook → n8n execution starts; sidecar files written
3	Waits	Worker creates <code>scans/testphp.vulnweb.com_<timestamp>/</code> , runs light + SQLi
4	Opens Dashboard → View	Reads vulnerabilities <code>.json</code> , severity badges, HTML report link
5	Optional	Reads AI pre/post markdown in <code>reports/</code>

Edge Cases

Case	Behaviour
Target outside scope	Portal error; no n8n execution
n8n down / workflow inactive	Portal shows engine offline or webhook 404
CLI tool missing	Recorded in scan JSON; scan continues
Rate limit / block detected	<code>.scan_stop</code> ; partial results retained; BLOCKED on dashboard

Section 09 — Target Users & Market

Primary User

Junior-to-mid **bug bounty hunters** and **pentest lab students** on Linux (Parrot/Kali) who want orchestrated recon with scope and guardrails—not another disconnected script folder.

Market Opportunity

Bug bounty and ASM markets continue to grow; solo hunters need local-first automation with program-safe defaults. Initial wedge: thousands of Linux-based researchers and university cyber labs.

Competitive Landscape

Alternative	Weakness vs. Cyber League
Manual script collections	No visual orchestration; weak scope model
Commercial ASM platforms	Costly; not local-first; overkill for labs

Alternative	Weakness vs. Cyber League
Raw n8n without pentest pack	No scope/SQLi/guard integration out of the box

Section 10 — Business Model

Revenue Model

Freemium / open core (future): free local runner; paid cloud for team dashboards, report branding, and compliance exports.

Pricing Hypothesis

\$19–49/month per researcher for cloud sync and team features; local runner remains free/open core.

Go-to-Market

1. Publish n8n workflow template to n8n community.
2. YouTube lab demos on authorized vulnweb targets (see demo video).
3. University cybersecurity clubs and CTF teams on Parrot/Kali.

Roadmap

Horizon	Deliverable
Now (hackathon)	Cyber League + n8n workflow + submission doc + demo video
0–3 months	Docker Compose, path-agnostic config, Burp export
12 months	Team cloud, program CSV import, authenticated scans

Section 11 — Why This Project Leads the Track

Technical Edge

- **Deep n8n usage:** webhook ingress, Code parsing, Execute Command worker, AI nodes, form fallback—not a thin wrapper.
- Workflow patch suite (`patch_*.py`) shows repeatable deploy after import.
- Portal ↔ n8n contract (JSON + sidecar files) addresses real n8n v2 constraints (`fs`, disabled Execute Command by default).
- Differentiators: **scope enforcement**, **SQLi phase**, **rate-limit guard** in one pipeline.

Problem–Solution Fit

Directly targets fragmented recon and out-of-scope probing with one visual pipeline testers can extend without recompiling the portal.

Execution Quality

- End-to-end demo: portal → n8n → artifacts → report → AI.
- Presets match real time budgets; polished local UI; `cyber league.sh` / `shutdown.sh` for operators.
- Recorded demo: <https://youtu.be/d1xiZMBJ61k>

Real-World Potential

Hunters can fork the workflow, add nuclei templates, plug Jira export. Fits n8n marketplace and local lab workflows beyond the hackathon.

Section 12 — Team & Roles

Team Members

Name	Role	Built	Background
Thanus	Builder	Portal (FastAPI), n8n	Linux

Name	Role	Built	Background
hiyan Kant	Lead	webhook integration, bugbounty-scan.sh, scope + SQLi modules, rate-limit guard, AI advisor scripts, workflow patches, ops scripts, documentation	bug-bounty to organizing and full-time student hacker interested in rat
ha	Member		ion
sa			on
mi	Lead		Pa
			rro
			t/K
			ali
			-
			st
			yl
			e
			w
			or
			ks
			tat
			io
			ns

*Hackers League also submitted Aivura (Project 2) on branch aivura —
Kiruththiyan Theiventhirarasa, team lead; demo https://t.me/Aivura_bot.*

Why This Team

Thanushiyan combines hands-on bounty recon experience with n8n orchestration skills—the right mix to wire a **real** pentest toolchain into the track sponsor's workflow engine with scope and safety defaults, not a demo-only chatbot.

*Cyber League · Hackers League · Cursor Buildathon · Cursor × TechTalk360 ·
Confidential*

Cyber League · Hackers League · Cursor × TechTalk360 · Confidential